

Building and securing REST API

1. Introduction to API Security

The MoMo SMS API exposes financial transaction records to client applications such as mobile and web apps. Because this data describes who paid whom, how much, and when, it must be protected. For this project, we set three security goals:

Goal	How we addressed it
Only authorized clients can read or modify data	HTTP Basic Authentication is checked on every endpoint. Requests with missing, malformed or wrong credentials receive 401 Unauthorized with the header <code>WWW-Authenticate: Basic realm="MoMo API"</code> .
Bad requests must not crash the server or corrupt the dataset	Input validation on every write: the JSON body must be valid, the fields <code>transaction_type</code> , <code>amount</code> , <code>sender</code> and <code>receiver</code> must be present, <code>amount</code> must be a number, and the <code>id</code> in the URL must be numeric.
Responses must be predictable and machine-readable	Every error is returned as JSON in the form <code>{"error": "..."} </code> with a standard HTTP status code (400, 401, 404), so client apps can handle failures reliably.

The server reads the username and password from the environment variables `API_USERNAME` and `API_PASSWORD` (with a development default), so they can be changed without editing the source. The data comes from `modified_sms_v2.xml`, which is parsed in Python into a list of JSON objects. Fields such as transaction type, amount, sender, receiver, fee, balance and transaction id are extracted from the SMS body text, since the XML stores them only inside a free-text string.

2. Documentation of endpoints

Base URL: `http://localhost:8000`

Authentication: HTTP Basic Auth on every endpoint.

Required fields for POST/PUT: `transaction_type`, `amount`, `sender`, `receiver` (amount must be a number).

Method	Endpoint	Purpose
GET	<code>/transactions</code>	List all transactions

GET	/transactions/{id}	View one transaction
POST	/transactions	Add a new transaction
PUT	/transactions/{id}	Update an existing transaction
DELETE	/transactions/{id}	Delete a transaction

2.1 GET /transactions - List all transactions

Request

```
curl -u admin:secret http://localhost:8000/transactions
```

Response - 200 OK (JSON array)

```
[
  {
    "id": 1,
    "transaction_type": "received",
    "amount": 2000,
    "fee": 0,
    "balance_after": 2000,
    "sender": "Jane Smith",
    "receiver": "You",
    "sender_phone": "*****013",
    "receiver_phone": null,
    "transaction_id": "76662021700",
    "timestamp": "2024-05-10 16:30:51",
    "date_epoch_ms": 1715351458724,
    "readable_date": "10 May 2024 4:30:58 PM",
    "body": "You have received 2000 RWF from Jane Smith ..."
  }
]
```

Error codes

Code	Meaning
401	Missing or invalid credentials

2.2 GET /transactions/{id} - Get one transaction

Request

```
curl -u admin:secret http://localhost:8000/transactions/1
```

Response - 200 OK

```
{
  "id": 1,
  "transaction_type": "received",
  "amount": 2000,
  "fee": 0,
  "balance_after": 2000,
  "sender": "Jane Smith",
  "receiver": "You",
  "sender_phone": "*****013",
  "receiver_phone": null,
  "transaction_id": "76662021700",
  "timestamp": "2024-05-10 16:30:51",
  "date_epoch_ms": 1715351458724,
  "readable_date": "10 May 2024 4:30:58 PM",
  "body": "You have received 2000 RWF from Jane Smith ..."
}
```

Error codes

Code	Meaning
401	Missing or invalid credentials
400	{"error": "Invalid ID"} - id is not a number
404	{"error": "Not Found"} - id does not exist

2.3 POST /transactions - Create a transaction

Request

```
curl -u admin:secret -X POST \
  -H "Content-Type: application/json" \
  -d '{"transaction_type": "payment", "amount": 500, "sender": "You", "receiver": "Test"}' \
  http://localhost:8000/transactions
```

Response - 201 Created

```
{
  "id": 1693,
  "transaction_type": "payment",
  "amount": 500,
  "sender": "You",
  "receiver": "Test"
}
```

Error codes

Code	Meaning
401	Missing or invalid credentials
400	{"error": "Invalid or missing JSON body"}
400	{"error": "Missing field: <name>"}
400	{"error": "amount must be a number"}

The id is assigned automatically (one greater than the current highest id).

2.4 PUT /transactions/{id} - Replace a transaction

Request

```
curl -u admin:secret -X PUT \  
-H "Content-Type: application/json" \  
-d '{"transaction_type": "payment", "amount": 9999, "sender": "You", "receiver": "Updated"}' \  
http://localhost:8000/transactions/1
```

Response - 200 OK

```
{  
  "id": 1,  
  "transaction_type": "payment",  
  "amount": 9999,  
  "sender": "You",  
  "receiver": "Updated"  
}
```

Error codes

Code	Meaning
401	Missing or invalid credentials
400	{"error": "Invalid or missing JSON body"}
400	{"error": "Missing field: <name>"}
400	{"error": "amount must be a number"}
404	{"error": "Not Found"} - id does not exist

2.5 DELETE /transactions/{id} - Delete a transaction

Request

```
curl -u admin:secret -X DELETE http://localhost:8000/transactions/1
```

Response - 200 OK

```
{"message": "Transaction 1 deleted"}
```

Error codes

Code	Meaning
401	Missing or invalid credentials
400	{"error": "Invalid ID"}
404	{"error": "Not Found"}

2.6 Test Evidence (Screenshots)

All endpoints were tested with curl. The screenshots are stored in the screenshots/ folder of the repository.

Screenshot 1: Successful GET with authentication



```
GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
$ curl -i -u admin:secret http://localhost:8000/transactions/1
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.3
Date: Mon, 28 Sep 2026 20:08:07 GMT
Content-Type: application/json
Content-Length: 544

{"id": 1, "transaction_type": "received", "amount": 2000, "fee": 0, "balance_after": 2000, "sender": "Jane Smith", "receiver": "You", "sender_phone": "*****013", "receiver_phone": null, "transaction_id": "76662021700", "timestamp": "2024-05-10 16:30:51", "date_epoch_ms": 1715351458724, "readable_date": "10 May 2024 4:30:58 PM", "body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}
GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
```

Screenshot 2: Unauthorized request with wrong credentials (401)

```
Problems Output Debug Console Terminal Ports

GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
● $ curl -i -u admin:wrong http://localhost:8000/transactions
HTTP/1.0 401 Unauthorized
Server: BaseHTTP/0.6 Python/3.14.3
Date: Mon, 28 Sep 2026 20:01:58 GMT
Content-Type: application/json
Content-Length: 25
WWW-Authenticate: Basic realm="MoMo API"

{"error": "Unauthorized"}
GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
○ $
```

Screenshot 3: Successful POST (201)

```
Problems Output Debug Console Terminal Ports

GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
$ curl -i -u admin:secret -X POST -H "Content-Type: application/json" -d '{"transaction_type": "payment", "amount": 500, "sender": "You", "receiver": "Test Person"}' http://localhost:8000/transactions
HTTP/1.0 201 Created
Server: BaseHTTP/0.6 Python/3.14.3
Date: Mon, 28 Sep 2026 20:09:06 GMT
Content-Type: application/json
Content-Length: 102

{"transaction_type": "payment", "amount": 500, "sender": "You", "receiver": "Test Person", "id": 1693}
GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
○ $
```

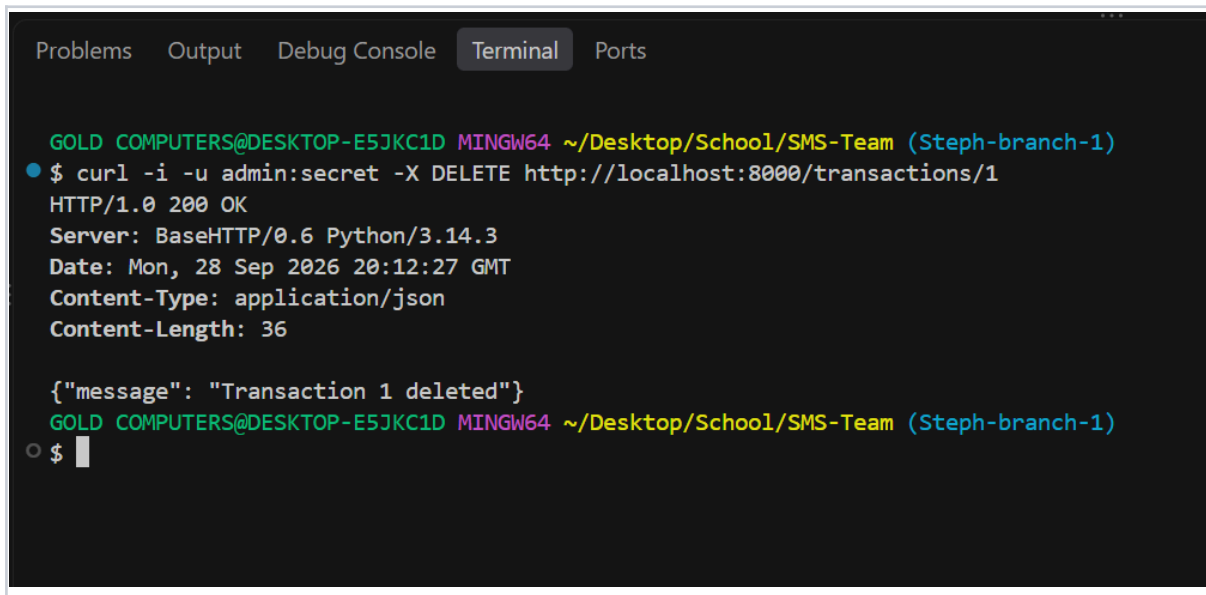
Screenshot 4: Successful PUT (200)

```
Problems Output Debug Console Terminal Ports

GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
● $ curl -i -u admin:secret -X PUT -H "Content-Type: application/json" -d '{"transaction_type": "payment", "amount": 9999, "sender": "You", "receiver": "Updated Person"}' http://localhost:8000/transactions/1
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.3
Date: Mon, 28 Sep 2026 20:11:53 GMT
Content-Type: application/json
Content-Length: 103

{"transaction_type": "payment", "amount": 9999, "sender": "You", "receiver": "Updated Person", "id": 1}
GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
○ $
```

Screenshot 5: Successful DELETE (200)



```
Problems  Output  Debug Console  Terminal  Ports

GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
• $ curl -i -u admin:secret -X DELETE http://localhost:8000/transactions/1
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.3
Date: Mon, 28 Sep 2026 20:12:27 GMT
Content-Type: application/json
Content-Length: 36

{"message": "Transaction 1 deleted"}
GOLD COMPUTERS@DESKTOP-E5JKC1D MINGW64 ~/Desktop/School/SMS-Team (Steph-branch-1)
○ $
```

3. Results of DSA Comparison

We compared three search methods for finding transactions: Linear search, Dictionary lookup, and Binary search. These tests were performed using the first 20 records and the full dataset of 1,691 transaction records.

```
=== Small set: 20 records (average over every ID, 2000 pass(es)) ===
Method          Avg per lookup (us)    Worst case (us)    Speedup vs linear
Linear search    0.680                  1.179              1.0x
Dictionary lookup 0.194                  0.258              3.5x
Binary search    0.842                  1.102              0.8x

=== Full dataset: 1691 records (average over every ID, 5 pass(es)) ===
Method          Avg per lookup (us)    Worst case (us)    Speedup vs linear
Linear search    47.854                 97.947             1.0x
Dictionary lookup 0.211                  0.258              227.3x
Binary search    2.147                  2.432              22.3x

Saved DSA results to /SMS-Team/docs/dsa_results.txt
```

DSA Results

The screenshot above shows the benchmark results for linear search, Dictionary lookup, and Binary search using 20 records and the full dataset of 1,691 records.

Why is Dictionary lookup faster ?

Dictionary lookup uses the transaction ID as a key, so it can find a transaction directly while linear search checks each record one by one which takes more time to find as the number records increases.

Alternative:

we can also use Binary Search because it is faster than Linear Search when the data is sorted.

4. Reflection on Basic Auth Limitations

HTTP Basic Authentication sends the username and password with every request, base64-encoded in the Authorization header. Base64 is an encoding, not encryption: anyone who intercepts the traffic can decode it with a single command. Our API is suitable for a learning project, but Basic Auth has several real weaknesses:

Weakness	Why it matters
Credentials are sent on every request	The more often the secret travels over the network, the more chances it has to be captured.
Base64 is not encryption	Without HTTPS/TLS, the password is readable by anyone on the network path. Basic Auth is only acceptable over HTTPS.
No expiry or revocation	A stolen password works until it is manually changed. There is no way to invalidate one session or one device.
No fine-grained permissions	Every authenticated user has the same access. There are no roles or scopes (for example read-only vs. delete).
Password stored on the client	Apps must keep the raw password to resend it, and browsers may cache it.
No protection against brute force	Nothing in the protocol limits repeated guessing; rate limiting must be added separately.

Stronger alternatives

Alternative	How it improves security
-------------	--------------------------

JWT (JSON Web Tokens)	The user logs in once with their password and receives a signed token with an expiry time. Later requests send the token instead of the password. Tokens can carry roles and expire automatically, so a leaked token has limited value.
OAuth 2.0	A delegated-authorization framework: a client gets a short-lived access token (and optionally a refresh token) from an authorization server, with scoped permissions. Users never share their password with the client app. This is the industry standard for third-party access.
API keys / mutual TLS	Per-client keys that can be rotated or revoked individually, or certificates that authenticate both sides of the connection, useful for server-to-server access.

Conclusion

For this assignment Basic Auth demonstrates the core idea of protecting endpoints and returning 401 for invalid credentials. In production, the API should run behind HTTPS, use token-based authentication such as JWT or OAuth 2.0, hash any stored passwords, and add rate limiting and logging.